

Exp: 07-Write the Python Program to implement BFS

Aim:

To write a python program to implement BFS.

Algorithm:

Step 1: Initialize a queue data structure and a set to keep track of visited nodes.

Step 2: Add the starting node to the queue and to the set of visited nodes.

Step 3: While the queue is not empty, dequeue the next node from the queue.

Step 4: For each neighbor of the dequeued node, if it has not been visited yet, add it to the queue and mark it as visited.

Step 5: Repeat steps 3-4 until the queue is empty.

Program:

```
from collections import deque

def bfs(adj_list, start, end):
    """
    Perform a breadth-first search in the given graph to find the shortest path
    between the start and end nodes.
    """
    queue = deque([(start, [start])])
    visited = set()

    while queue:
        node, path = queue.popleft()
        if node == end:
            return path
        if node in visited:
            continue
```

```

        visited.add(node)
    for neighbor in adj_list[node]:
        if neighbor not in visited:
            queue.append((neighbor, path + [neighbor]))
    return None

if __name__ == '__main__':
    graph = {0: [1, 2],
              1: [0, 2, 3],
              2: [0, 1, 3],
              3: [1, 2, 4],
              4: [3]}
    start_node = 0
    end_node = 4
    shortest_path = bfs(graph, start_node, end_node)
    if shortest_path is not None:
        print(f"The shortest path between {start_node} and {end_node} is {shortest_path}")
    else:
        print(f"There is no path between {start_node} and {end_node}")

```

Output:

```

# Copy
The shortest path between 0 and 4 is [0, 1, 3, 4]
|

```

Result:

The Breadth-First Search (BFS) algorithm was successfully implemented to find the shortest path between two nodes in a graph.